

Distributed Search Engine for Accessible Digital Libraries

BLM4821 - Big Data Processing and Analysis | 21011916 Murat Ağuş

I. INTRODUCTION (MOTIVATION & GOALS)

Visually impaired individuals face significant barriers searching large digital libraries due to sequential reading constraints of audiobooks and screen readers. This project addresses this by developing a scalable, high-performance distributed search engine using Hadoop. By processing Project Gutenberg book corpora through a MapReduce pipeline, the system computes TF-IDF scores to build an inverted index, enabling real-time, ranked search results for assistive technologies.

II. SYSTEM ARCHITECTURE AND DATA FLOW

The system consists of data storage, distributed processing, and visualization layers. The network data flow architecture is summarized in Fig. 1.

The client application (`LibraryGUI`) communicates with the NameNode and DataNodes directly via the WebHDFS REST API, without requiring local Hadoop binaries or libraries. The Java application, running on the Windows host, interacts with a multi-node Hadoop 3.2.1 cluster (1 NameNode, 2 DataNodes, 1 ResourceManager, 2 NodeManagers, 1 HistoryServer) running inside Docker containers using HTTP requests.

III. USER INTERFACE AND USE CASE SCENARIOS

The client application (`LibraryGUI`) provides three primary panels to allow users to interact with the HDFS storage, run MapReduce indexing pipelines, and query the index.

1) HDFS Manager Panel (Dataset Management Scenario): The administrator needs to upload raw book files to HDFS and inspect remote directories. The administrator opens the HDFS Manager tab (Fig. 2), which triggers a WebHDFS `LISTSTATUS` request to render the directory structure. Clicking “Upload File” chooses a local file and issues a WebHDFS `CREATE` redirect call to a DataNode to stream the data. The tree list is dynamically refreshed showing metadata such as file size, replication factor, and block size. Remote `GET` and `DELETE` operations are supported to download or delete files.

2) MapReduce Job Monitor Panel (Pipeline Execution Scenario): The user needs to run the inverted index pipeline. The user navigates to the Job Monitor tab (Fig. 3), selects a corpus size (e.g., Small, Half, All), and triggers “Run Pipeline”. The GUI initiates the three sequential MapReduce jobs by submitting the JAR package on YARN. A background polling thread queries YARN ResourceManager and JobHistory APIs to retrieve the execution progress, updating real-time progress bars for Map and Reduce phases while streaming console output logs into the console text area.

3) Search Interface Panel (Ranked Query & Reader Scenario): A visually impaired user wants to search books

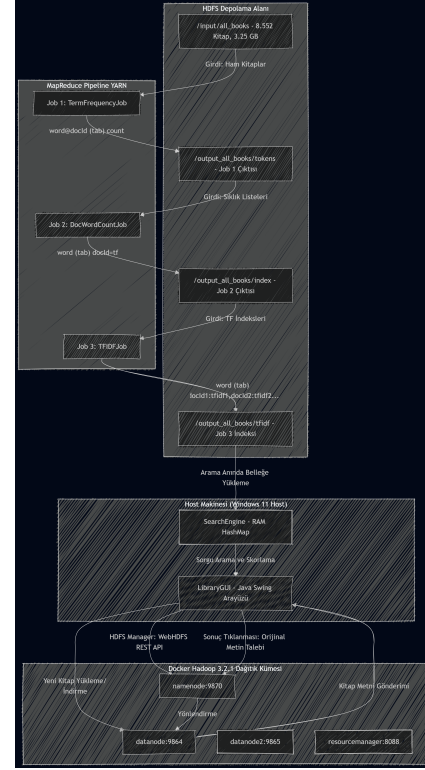


Figure 1. System Architecture and WebHDFS / YARN Data Flow Diagram.

containing specific words and read their contents. The user opens the Search tab (Fig. 4), selects the dataset index, and types a query (e.g., “wizard of oz”). The GUI processes the in-memory inverted index using a fast TF-IDF ranked search, displaying relevant book results sorted by relevance. The user clicks a book title, triggering an asynchronous WebHDFS request to fetch text. The GUI displays the text and highlights occurrences, allowing the user to navigate matching terms via hotkeys.

IV. MAPREDUCE PIPELINE JOBS

Three sequential MapReduce jobs are chained (`JobPipeline.java`) to calculate the inverted index and TF-IDF scores:

Job 1: Term Frequency (TermFrequencyJob): Tokenizes documents, cleans non-alphanumeric tokens (`[^a-zA-Z0-9]+`), and extracts docId from filenames.
Mapper: Input `<offset,line>`, Output `<word@docId,1>`.
Reducer: Output `<word@docId,count>`.

Job 2: Document Word Count Normalization (DocWordCountJob): Normalizes term frequencies: $TF = \text{word_count} / \text{total_doc_words}$.

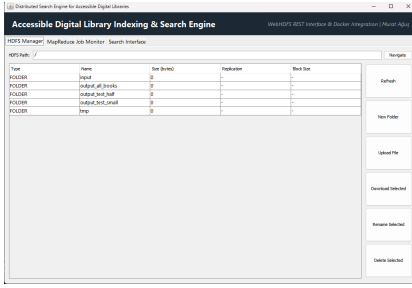


Figure 2. HDFS Manager Screen.

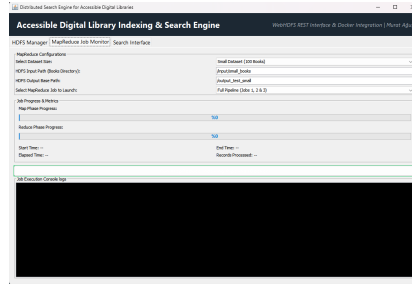


Figure 3. MapReduce Job Monitor.

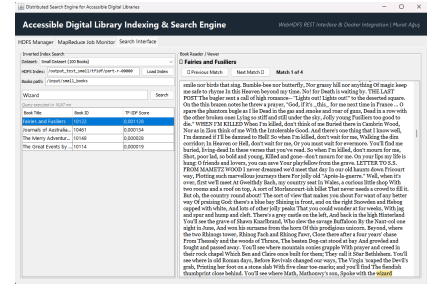


Figure 4. Ranked Search Screen.

Mapper: Input $\langle \text{line_no}, \text{word@docId} \setminus \text{t count} \rangle$, Output $\langle \text{docId}, \text{word=count} \rangle$. **Reducer:** Output $\langle \text{word}, \text{docId=TF} \rangle$.

Job 3: TF-IDF Scoring and Inverted Index (TFIDFJob): Counts Document Frequency (DF) and calculates $TF \cdot IDF = TF \times \ln(N/DF)$.

Mapper: Input $\langle \text{line_no}, \text{word} \setminus \text{t docId=TF} \rangle$, Output $\langle \text{word}, \text{docId=TF} \rangle$. **Reducer:** Output $\langle \text{word}, \text{docId1:tfidf1}, \text{docId2:tfidf2}, \dots \rangle$.

V. IMPLEMENTATION ENVIRONMENT

The system runs in a hybrid environment. The client GUI (LibraryGUI) runs on a Windows 11 Pro host with Java 24. The Hadoop YARN cluster runs inside Docker containerized Ubuntu 22.04 LTS environments, consisting of 7 active containers: 1 NameNode, 2 DataNodes, 1 ResourceManager, 2 NodeManagers, and 1 JobHistoryServer. Apache Hadoop 3.2.1 and Java 8 are installed inside the cluster. Project compilation is managed via Maven 3.9.6, and cluster REST calls are handled using a custom WebHDFS client.

VI. PERFORMANCE EVALUATION

The system was evaluated using three dataset sizes (Small: 100 books, Half: 4,000 books, All: 8,552 books). The execution times of the MapReduce jobs are presented in Table I.

Table I
MAPREDUCE PIPELINE EXECUTION TIMES

Dataset Size	Data Volume	Job 1 Time	Job 2 Time	Job 3 Time
Small (100)	37.6 MB	2m25s	13s	15s
Half (4,000)	1.43 GB	1h26m41s	1m3s	1m4s
All (8,552)	3.03 GB	3h5m26s	1m47s	2m7s

Job 1 (Term Frequency count) takes more than 85% of the total execution time because it parses all raw text files. Once the index file is loaded from HDFS into memory, the query performance metrics for the search query “wizard” are shown in Table II.

VII. DISCUSSION: LIMITATIONS OF MAPREDUCE IN SEARCH PHASE

The MapReduce programming model is designed for batch processing rather than real-time interactive search queries. First, **High Latency** is introduced due to container allocations, JVM startup overheads, and YARN resource management,

Table II
SEARCH AND MEMORY PERFORMANCE

Metric	Small Index	Half Index	All Index
Unique Terms	103,850	1,149,000	1,690,388
Index Loading Time	~1.2 sec	~5.8 sec	~8.4 sec
RAM Usage (JVM)	~92 MB	~1.1 GB	~1.75 GB
Query Response Time	0.11 ms	0.91 ms	1.99 ms

causing at least a 10-15 second delay. Second, **Disk I/O Overhead** is high since MapReduce writes intermediate data to HDFS disks, whereas online search requires pure RAM operations. Third, in **Production Environments**, MapReduce is used offline for batch calculations, while online queries are handled by specialized search engines like Elasticsearch, Solr, or Lucene.

VIII. TECHNICAL CHALLENGES AND LESSONS LEARNED

During system development, three key challenges were resolved: **1) WebHDFS NAT Redirection:** Docker NameNodes redirect requests using container-internal IPs. This was resolved in WebHDFSClient by translating paths dynamically using a redirection mapper (translateLocation) to host ports (9864, 9865). **2) Turkish Locale Bug:** Lowercase string conversions on Turkish host systems mapped “ı” to “i”, throwing regex errors. Using java.util.Locale.ROOT resolved this issue globally. **3) JVM Memory Overhead:** Compiling and loading 4,000+ files individually caused JVM freezing. This was resolved via dynamic sub-setting inside the driver and uploading the raw books directory as a single stream.

IX. CONCLUSION

The distributed search engine successfully processed large-scale text files in a multi-node Hadoop MapReduce cluster, generating a TF-IDF inverted index. Integrating WebHDFS and a Java GUI enabled visually impaired users to query books in milliseconds and read contents without access errors.